

ClawGym II: Exploring Black-Box RL on Agent Harness

Huatong Song^{1*}, Fei Bai^{1*}, Ming Yang², Renyuan Li², Jia Deng¹, Jujie He², Zhange Zhang²,
Daixuan Cheng¹, Yan Xing², Qi Yun², Xuxing Chen², Danyang Li², Feng Chang², Chuan Hao²,
Ran Tao², Jian Yang², Bryan Dai², Wayne Xin Zhao^{1†}, Mingjie Tang^{2†}, Ji-Rong Wen¹

¹Gaoling School of Artificial Intelligence, Renmin University of China,

²IQuest Research

*Equal contributors †Corresponding Authors

Email: {songhuatong123, feibai}@ruc.edu.cn,
batmanfly@gmail.com

Abstract

Agent harnesses have substantially improved performance on long-horizon tasks by coordinating agent interactions with the environment. However, reinforcement learning through complex harnesses remains largely unexplored, as scaling such training to long-horizon agent tasks introduces fundamental challenges. In this work, we present a **unified black-box RL framework** for stable and scalable optimization of general agents through complex harnesses. Concretely, we first build a sandbox-based execution infrastructure that isolates task environments and harnesses within temporary sandboxes for large-scale concurrent rollouts. We then decouple policy optimization from opaque harness execution and place a serving proxy at the model boundary to capture model calls. To reconstruct multi-turn trajectories and improve training efficiency, we organize the captured calls into prefix trees and further adapt both critic-based PPO and critic-free GRPO to optimize over the recovered tree structure. Meanwhile, we maintain training–inference consistency throughout the optimization process. Finally, we introduce **mix-harness training**, allowing a single model to be jointly optimized by heterogeneous harnesses. With Qwen3-30A3B, black-box RL improves Pass@1 on ClawGym-Bench by 9.98 and 14.81 points through **OpenClaw** and **Claude Code**, respectively, while remaining stable over 200–400 optimization steps. Moreover, the framework yields consistent gains on more challenging tasks such as JobBench and OfficeQA. Overall, our framework enables effective, stable, and scalable optimization of general agents through black-box harnesses, supporting unified training across heterogeneous execution systems.

1. Introduction

Agent harnesses [30] have emerged as the foundational operating layer of autonomous agent systems, coordinating the interaction between large language models [38] and their environments throughout the task-solving process. They typically integrate system prompts, tool interfaces, context management, workflow orchestration, and retry and recovery mechanisms into a unified runtime. Production-grade systems such as Claude Code [2], Codex [17], and OpenClaw [19] demonstrate that carefully designed harnesses can substantially improve agent performance on complex long-horizon tasks [24].

As these harnesses become increasingly capable, they are accelerating the development of general autonomous agents that operate across diverse real-world workspaces and solve tasks spanning software engineering [7, 16], office applications [8, 21], AI assistant workflows [23, 36], and MCP-enabled tool use [5, 13]. Recent progress has been driven by inference-side advances in harness design, optimization, and self-evolution [31]. Yet the benefits of increasingly capable harnesses depend not only on their design but also on how effectively the underlying model can leverage them. Even a strong model may

perform poorly within a capable harness if it has not been suitably trained, motivating efforts, typically via reinforcement learning (RL) algorithms, to enable models to better exploit these agent harnesses.

A defining characteristic of modern harnesses is their inherent complexity: the design and implementation are sufficiently intricate. As a result, RL training algorithms targeting a harness can differ substantially from those used in fully textual [6] or otherwise simple environments [27, 28]. In simpler single-turn or lightweight agentic settings, the interaction process is explicit and directly observable, allowing RL to optimize well-defined model trajectories without accounting for complex hidden execution logic. With modern harnesses, by contrast, the mainstream approach [11, 32] is to treat the internal control flow and execution logic as opaque and to design a black-box RL algorithm. Indeed, black-box RL has become a key technical cornerstone for developing and leveraging capable harnesses for general agents [37]. Yet stable and scalable policy optimization for general agent tasks through such harnesses remains relatively underexplored in the existing literature. This work therefore focuses on the following research question:

How can black-box RL be performed through a harness to stably optimize general agents?

To answer this question, we center our investigation on three fundamental challenges:

- *Scalable and stable infrastructure for black-box RL.* General agent tasks require dedicated, stateful environments during RL, and large-scale concurrent rollouts place substantial pressure on the execution infrastructure. For long-horizon tasks, runtime delays and failures can accumulate across interaction steps and invalidate entire trajectories, making timely and reliable data generation difficult at scale.
- *Efficient and effective policy optimization from fragmented black-box traces.* Black-box harnesses expose fragmented, forked, and often redundant model calls rather than complete interaction trajectories, making these traces difficult to use directly for reinforcement learning. Moreover, opaque internal operations can further amplify training–inference inconsistency, leading to unstable training.
- *Extensibility across heterogeneous harnesses.* Harnesses differ substantially in their interaction protocols, tool interfaces, context management, and execution workflows. A practical framework should support diverse harnesses with minimal system-specific adaptation and enable unified training across them, rather than optimizing policies for a single execution system.

To address these challenges, we develop a **unified black-box RL framework** to stably optimize general agents through any complex harness. First, to sustain stable, large-scale concurrent environment execution during rollout, we build an execution and data infrastructure that preserves faithful model behavior. Each task-specific environment and its execution harness are encapsulated within a temporary sandbox that is provisioned on demand and destroyed upon completion, providing an isolated workspace for every rollout while enabling concurrency at scale. Second, we decouple policy optimization from harness execution by treating the harness as an opaque rollout engine, while a serving proxy intercepts every model call. This design allows the harness to retain its native execution logic and operate independently. To recover reliable multi-turn training trajectories and improve training efficiency, we organize the captured calls from each rollout into a **prefix tree**. We then adapt both critic-based PPO [25] and critic-free GRPO [26] to optimize over the recovered tree structure. Training–inference consistency is further preserved through the **black-box token-in-token-out** mechanism [9] and token-level importance-sampling rollout correction [34], which mitigate biases arising from differences between the underlying inference and training engines. Additional safeguards stabilize both rollout execution and policy optimization. Finally, encapsulating each harness within the sandbox isolates harness-specific execution from the training pipeline, allowing different harnesses to be readily selected or replaced for different task environments. Building on this modular design, we introduce **mix-harness training**, where each task–harness pair forms a basic training instance and rollouts from multiple heterogeneous harnesses are jointly used to optimize a single model.

We validate the black-box RL framework on two representative and structurally different harnesses:

OpenClaw, a general-purpose harness for personal-assistant tasks, and **Claude Code**, a mature harness designed for long-horizon code and terminal interaction. General agent tasks from ClawGym [4] are used for training under both harnesses. Starting from the Qwen3-30A3B [33] backbone, black-box RL training produces ClawII-OC-30A3B (trained through OpenClaw) and ClawII-CC-30A3B (trained through Claude Code), improving Pass@1 on PinchBench by 11.71 and 17.28 points and on ClawGym-Bench by 9.98 and 14.81 points, respectively. Additionally, training remains stable across 200–400 optimization steps, demonstrating the effectiveness and robustness of our black-box RL framework for optimizing general agent tasks on complex harnesses. Furthermore, under mix-harness training, the trained model matches or even surpasses the performance of its counterparts trained with either harness alone, demonstrating that black-box RL can seamlessly integrate heterogeneous harnesses within our unified training pipeline. Beyond these settings, we extend our framework to more challenging task settings from JobBench [14] and OfficeQA [20], where black-box RL continues to deliver consistent improvements.

Our main contributions are summarized as follows:

- We establish a unified and extensible black-box RL framework for general agents that accommodates training through any individual opaque harness and further enables mix-harness training via joint optimization across heterogeneous harnesses.
- We develop the infrastructure and optimization techniques for stable and scalable black-box RL, supporting reliable concurrent rollouts, faithful trajectory recovery, tree-structure optimization with PPO and GRPO, and training–inference consistency.
- We validate the framework on OpenClaw and Claude Code, demonstrating consistent performance gains and stable optimization, together with its extensibility across heterogeneous harnesses through mix-harness training and to more challenging tasks such as JobBench and OfficeQA.

2. Preliminaries

2.1. Problem Formulation for General Agent Tasks

A general agent task consists of a user instruction together with an isolated, stateful environment in which the task is executed. The environment is initialized from a workspace and provides the execution context, including files, directories, documents, software repositories, running services, and other task-specific resources. To accomplish the task, the agent interacts with this environment over multiple turns through external tools, such as web search, file editing, code execution [3], shell commands, or API calls, progressively transforming its state toward the desired goal. The outcome is reflected in the final environment state, typically through generated artifacts (e.g., documents, presentations, or reports) or successfully completed operations (e.g., fixing a software repository or configuring a system). We denote a task as $q = (u, \mathcal{W}_0)$, where u is the user instruction and $\mathcal{W}_0$ is the initial workspace, and use $\mathcal{E}_q$ to denote the associated execution environment initialized from $\mathcal{W}_0$, with tasks sampled from $q \sim \mathcal{D}$.

During execution, the agent alternates between observations and actions and produces a trajectory

$$\tau = (o_1, a_1, \dots, o_T, a_T),$$

where o_t denotes the observation at step t and a_t denotes the corresponding agent action. The interaction terminates when the task is completed or execution stops, resulting in a final workspace state $\mathcal{W}_T$. This final state is then evaluated to obtain a rollout-level reward.

In practice, task evaluation typically follows two paradigms. Rule-based rewards verify the correctness of the final workspace through deterministic procedures, such as unit tests, program execution, or exact-match validation. Rubric-based rewards rely on a stronger evaluator (e.g., an LLM-as-a-Judge) to assess the overall quality of the generated artifact or final workspace according to task-specific evaluation criteria.

2.2. Harness-Driven Agent Execution

Modern general-purpose agents are increasingly deployed through mature harnesses, such as OpenClaw, Claude Code, and Codex. Compared with traditional handcrafted agent loops based on the standard ReAct [35] paradigm, these harnesses [1, 10] provide a substantially richer execution abstraction between the language model and the external workspace. They integrate capabilities such as tool orchestration, context management, built-in skills, subagent delegation, and failure recovery into a unified runtime system, enabling agents to solve substantially more complex long-horizon tasks while abstracting away many low-level execution details from the model.

Under this execution paradigm, the model no longer interacts directly with the workspace. Instead, given a model action a_t , the harness executes the corresponding tools, updates the workspace and execution context, and returns the processed observation for the next decision. This interaction can be abstracted as

$$(\mathcal{W}_{t+1}, o_{t+1}, c_{t+1}) = \mathcal{H}(\mathcal{W}_t, c_t, a_t), \quad (1)$$

where $\mathcal{H}$ denotes the harness, $\mathcal{W}_t$ is the workspace state, c_t is the maintained execution context, and o_{t+1} is the observation returned to the model. Consequently, the interaction trajectory is determined jointly by the model and the harness rather than by the model alone, making the harness the fundamental execution abstraction for modern long-horizon agent systems.

2.3. Agentic Reinforcement Learning Algorithms

Two mainstream reinforcement learning paradigms are investigated, distinguished primarily by their reliance on an explicit value model: Proximal Policy Optimization (PPO) as a critic-based method, and Group Relative Policy Optimization (GRPO) as a critic-free method. Throughout, we let the state s_t denote the interaction history $(o_1, a_1, \dots, o_t)$ that precedes action a_t , so that the policy $\pi_\theta(a_t | s_t)$ and the value function $V_\phi(s_t)$ are both defined over this history.

2.3.1. Proximal Policy Optimization (PPO)

The PPO algorithm optimizes the policy model $\pi_\theta(a|s)$ in conjunction with an auxiliary value model $V_\phi(s)$ that estimates the state-value function. To exclude environment-generated and invalid tokens from policy optimization, a binary mask $M_t \in \{0, 1\}$ is applied. The resulting masked clipping objective is

$$\mathcal{L}_{\text{PPO}}(\theta) = \hat{\mathbb{E}}_t \left[M_t \cdot \min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (2)$$

where $\rho_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$ denotes the token-level policy probability ratio, and $M_t = 1$ for valid policy-generated tokens and 0 otherwise.

The same mask is incorporated into Generalized Advantage Estimation (GAE) to prevent invalid positions from contributing to temporal credit assignment:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \cdot M_{t+l}, \quad (3)$$

where $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ denotes the temporal-difference residual. The critic network is optimized by minimizing the masked mean-squared loss:

$$\mathcal{L}_{\text{Value}}(\phi) = \hat{\mathbb{E}}_t \left[M_t \cdot (V_\phi(s_t) - G_t)^2 \right], \quad (4)$$

where G_t is the masked discounted return.

2.3.2. Group Relative Policy Optimization (GRPO)

To avoid the memory overhead and training complexity of a separate value model $V_\phi(s)$, GRPO estimates advantages relative to a group of sampled rollouts. For a given task q , the policy model π_θ generates a group of G independent trajectories $\{\tau_1, \tau_2, \dots, \tau_G\}$ with corresponding rewards $\{R_1, R_2, \dots, R_G\}$. The advantage $\hat{A}_i$ of the i -th trajectory is calculated directly from the group rewards:

$$\hat{A}_i = \frac{R_i - \mu_q}{\sigma_q + \epsilon}, \quad \mu_q = \frac{1}{G} \sum_{j=1}^G R_j, \quad \sigma_q = \sqrt{\frac{1}{G} \sum_{j=1}^G (R_j - \mu_q)^2}. \quad (5)$$

Reusing the token-level mask and policy ratio introduced above, with an additional trajectory index, the masked GRPO objective is

$$\mathcal{L}_{\text{GRPO}}(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{\sum_{t=1}^{|\tau_i|} M_{i,t}} \sum_{t=1}^{|\tau_i|} M_{i,t} \left[\min(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i) - \beta \mathbb{D}_{\text{KL}}(\pi_\theta(\cdot | s_{i,t}) || \pi_{\text{ref}}(\cdot | s_{i,t})) \right]. \quad (6)$$

Here, $M_{i,t}$ and $\rho_{i,t}$ are the trajectory-indexed counterparts of M_t and ρ_t , respectively. π_{ref} is the reference policy used to constrain policy drift, and $\mathbb{D}_{\text{KL}}$ is the Kullback–Leibler divergence.

3. Unified Black-Box RL through Complex Agent Harnesses

Our black-box RL formulation directly optimizes a trainable model through an unmodified and opaque agent harness, while decoupling policy optimization from harness execution. This section presents the execution infrastructure for scalable black-box rollouts, the recovery and optimization of training trajectories from captured model calls, and the extension to training across heterogeneous harnesses.

3.1. Infrastructure for Scalable Black-Box RL Execution

Black-box RL for general agent tasks requires large-scale concurrent execution of dedicated, stateful environments. Unlike stateless generation, each rollout may modify files, processes, tool states, and other task-specific resources over many interaction steps, making isolated execution essential. For each task q , we initialize a task-specific environment from its initial workspace $\mathcal{W}_0$ and launch the selected harness inside a temporary sandbox. The sandbox provides an isolated workspace and the runtime dependencies required by the harness, preventing interference across concurrent rollouts. Each sandbox is provisioned at rollout start and released after completion, allowing execution to scale with available sandbox capacity. In addition to isolating task states, each sandbox provides the external capabilities required during rollout. Beyond the basic tools supplied natively by the selected harness, common capabilities (e.g., web search), together with task-specific tools, are exposed through Model Context Protocol (MCP) servers launched inside the sandbox. Standardized MCP interfaces ensure reliable access to these services throughout execution and allow new capabilities to be integrated without modifying the native harness workflow.

We decouple policy optimization from harness execution. The training engine is responsible for policy optimization, while the inference engine serves the current policy during rollout. The harness itself drives the interaction inside the sandbox and retains control over tool use, context management, retries, and environment interaction. This separation avoids reimplementing harness-specific control flow in the training pipeline and enables different black-box harnesses to be integrated with minimal adaptation.

Although the internal control flow of the harness is inaccessible, model behavior remains observable at the model-serving boundary. Every model-generated action is produced through a model request that crosses this boundary. We therefore place a serving proxy at the boundary as the model endpoint used by

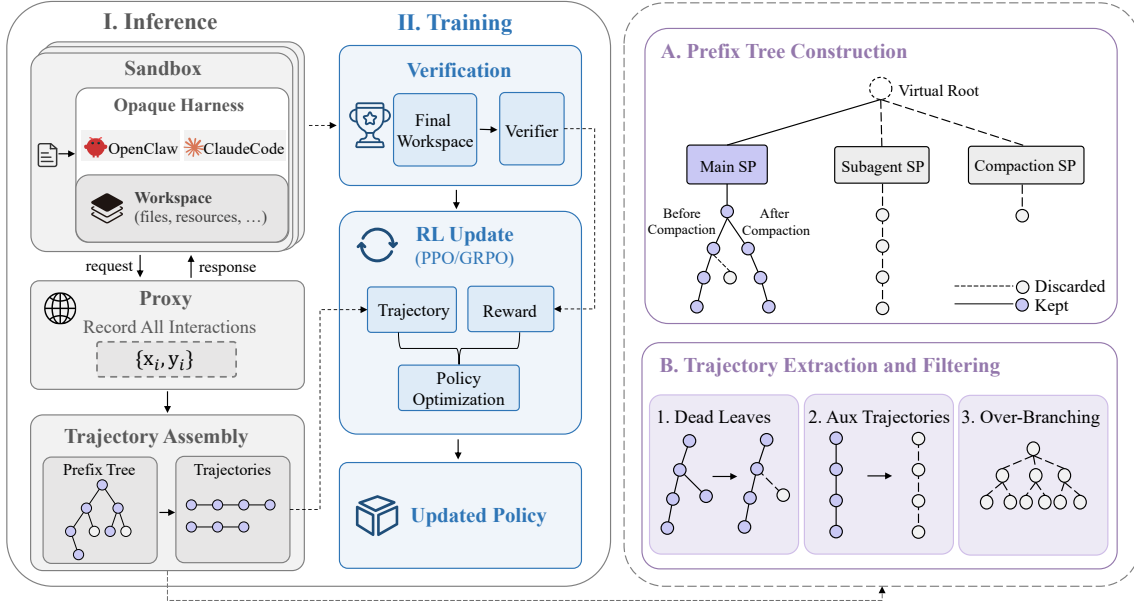


Figure 1. Overview of our black-box RL framework for optimizing general agents through harnesses.

the harness. For each request, the proxy invokes the current rollout policy, returns the generated response in the protocol expected by the harness, and records the exact input tokens, generated tokens, rollout log-probabilities, and associated task metadata. This boundary-level capture provides the model-side information required for optimization without instrumenting the internal harness logic.

As illustrated in Figure 1, each rollout launches the task environment and harness inside a sandbox, after which all model requests pass through the serving proxy. Upon completion, the verifier evaluates the final workspace state and produces a rollout-level reward. The captured model-call records and reward are passed to the training pipeline, where the calls are reconstructed into trainable multi-turn trajectories, as described in Section 3.2. The training engine then updates the policy model and synchronizes the parameters to the inference engine for the next iteration.

3.2. Bridging Black-Box Harness Execution and Policy Optimization

The serving proxy captures the model-side records required for optimization, but these records must first be transformed into structured multi-turn training trajectories. We organize the captured calls into prefix trees, optimize PPO and GRPO over the recovered tree structure, and preserve training–inference consistency throughout policy optimization.

3.2.1. Prefix Tree Construction

The model calls captured during a black-box rollout are fragmented, forked, and potentially redundant. Optimizing these calls independently would destroy the long-horizon interaction structure and repeatedly train on shared histories that recur across successive calls. We therefore organize the model calls captured from each rollout into a *rollout-level prefix tree* [12], which reconstructs their shared interaction structure and provides the training representation over which policy optimization is performed.

Concretely, consider a rollout that produces model calls $C = \{(x_i, y_i)\}_{i=1}^m$, where x_i is the input context of the i -th call and y_i is the corresponding model response. Within an uninterrupted interaction segment, later inputs extend histories established by earlier calls. Context compaction or subagent execution may start a new segment from a shortened or alternative context, after which subsequent calls continue to extend the newly established history. We construct a tree T rooted at the initial task prompt and attach each call to the

existing node whose accumulated history forms the longest prefix of x_i . By comparing the input of a child call with the accumulated parent history and the recorded parent response, we also recover the intervening non-model content introduced by the harness, including tool outputs and other environment feedback.

A leaf is a call that no later call extends, and each root-to-leaf path defines a candidate multi-turn trajectory. Mature harnesses may introduce multiple leaves through context management or subagent scheduling, which start new interaction branches instead of continuing the current one. By representing shared interaction histories with common tree nodes, the prefix tree stores each shared prefix only once while preserving distinct continuations as separate branches. The resulting leaves are processed as described in Section 3.2.2, and the filtered tree is subsequently used for policy optimization as detailed in Section 3.2.3.

3.2.2. Trajectory Extraction and Filtering

The prefix tree represents the model-call structure exposed during the task-solving process, and tracing a valid leaf back to the root recovers a candidate multi-turn training trajectory. However, not every leaf corresponds to the completion of the main task-solving process, as some arise from fault handling, context management, or subagent processes. We filter the resulting trajectories to retain those suitable for training.

Removing the Dead Leaves. A single turn may be generated multiple times when the inference server retries a request or the harness regenerates an invalid response, as OpenClaw does after a malformed tool call. Superseded attempts receive no further continuation but remain in the prefix tree as dead leaves. We partition the candidate trajectories into interaction segments delimited by context-compaction points and, within each segment, retain the leaf with the longest valid continuation. Shorter sibling trajectories terminating at retry-induced dead leaves are discarded.

Discarding Over-Branching Tasks. When the prefix tree of a rollout branches into an excessive number of leaves, the rollout has typically entered repeated or failed generation rather than legitimate interaction branching, and the resulting trajectories provide little useful training signal. Once the number of leaves exceeds a preset threshold, we discard the corresponding rollout, sacrificing a small amount of data to prevent corrupted signals from entering the gradient update.

Excluding Auxiliary Trajectories. Subagent and compaction trajectories are excluded because their roles in the harness differ from the role of the main task-solving trajectory and their contributions are not directly aligned with the rollout-level task reward. Broadcasting the same terminal reward to these auxiliary interactions would introduce ambiguous credit assignment and noisy optimization signals, potentially destabilizing training. We therefore optimize only the main-agent trajectories and leave effective use of auxiliary interactions to future work.

3.2.3. Policy Optimization over the Tree Structure

Once the prefix tree is constructed and invalid leaves are filtered, a single rollout may retain multiple valid root-to-leaf trajectories that share portions of the interaction history. Rather than collapsing them into a single trajectory, we retain this multi-trajectory structure and optimize the policy over the recovered tree. Since each rollout is evaluated once based on its final workspace state, all trajectories recovered from the same rollout inherit the same terminal reward. During optimization, all retained trajectories jointly contribute to the policy loss over the recovered tree. Trainable token nodes on branch-specific continuations are included normally, while nodes in shared prefixes are counted only once per rollout, preventing more highly branched rollouts from receiving disproportionate optimization weight.

Formally, we denote by $\{g_1, \dots, g_n\}$ the n rollouts of a task q , where rollout g_i contains trajectories $\{\tau_{i,1}, \dots, \tau_{i,m_i}\}$ and is assigned a terminal reward R_i shared across them. We next describe how GRPO and PPO are adapted to optimize over this multi-trajectory structure. The key adaptation is to preserve

the rollout-level reward semantics while allowing all reconstructed trajectories from the same black-box rollout to contribute to policy optimization.

Advantage Estimate for GRPO. GRPO naturally accommodates this structure: it estimates advantages from a group of rollouts for the same task and does not require a value model. We take the group to be the n rollouts of q and normalize each rollout’s reward against the group statistics,

$$\hat{A}_i = \frac{R_i - \mu_q}{\sigma_q + \varepsilon}, \quad \mu_q = \frac{1}{n} \sum_{j=1}^n R_j, \quad \sigma_q = \sqrt{\frac{1}{n} \sum_{j=1}^n (R_j - \mu_q)^2}, \quad (7)$$

where ε guards against division by zero. The advantage $\hat{A}_i$ is computed once per rollout and assigned to all trainable token nodes covered by the retained trajectories in g_i , with tokens in shared prefixes contributing to the loss only once.

Advantage Estimate for PPO. PPO employs an additional value model V_ϕ to estimate token-level advantages and jointly optimizes the policy and value models. Fully modeling the dependencies among forked trajectories would require more complex credit assignment over the recovered tree structure. We therefore adopt a simplified variant of PPO in which trajectories within the same rollout are treated independently, with $\gamma = 1$ and $\lambda = 1$.

Specifically, each trajectory $\tau_{i,j}$ receives the reward R_i assigned to its rollout at its own final token and performs GAE backup independently. No advantage signal is propagated across branch points connecting sibling trajectories. Under this setting, the standard GAE recursion degenerates to

$$\hat{A}_t = R_t - V_\phi(s_t), \quad (8)$$

where the advantage is only related to the final reward and the value estimate at the current state. This simplification requires the value model to estimate long-horizon returns from intermediate states and may increase advantage variance because no temporal discounting is applied. A more principled treatment of forked trajectories is left for future work.

3.2.4. Training–Inference Consistency

During RL, the policy model must be trained on the exact token sequence generated during rollout, so that the sequence presented to the training engine matches the one actually sampled by the policy. In the black-box setting, this requirement is complicated by harness-side transformations, such as tool-call normalization and assistant-message re-serialization, which may alter the representation of model outputs before they re-enter subsequent interactions. Simply re-tokenizing the reconstructed root-to-leaf trajectories would encode the harness-transformed model responses, producing token sequences that may differ from those originally sampled during rollout.

To keep the trained sequence identical to the generated one, we adopt the *black-box token-in-token-out* discipline [9] that maintains two decoupled views of every model call. The tokens generated by the inference engine are grafted directly onto the prefix tree and constitute the sole source of training data while the structured text that the harness receives is decoded from those same tokens purely for the harness to act on, and is never encoded back into the trained trajectory. Once a tool call is completed, the resulting environment response is encoded into a token sequence exactly as presented in the next model request and appended to the prefix tree. The two views thus serve different consumers: the harness drives the interaction from the decoded text, while training reads the generated tokens verbatim. Regardless of how the harness reformats or standardizes what it displays and feeds into later turns, those edits stay on its own view and cannot perturb the token record, so the sequence handed to the training engine is by construction identical to the one the policy model sampled.

Exact token sequences are insufficient, since the probability under which a token was sampled can differ from the one attributed to it at training time. The inference engine realizes the rollout policy π_{rollout} and records the log probability of each sampled token, whereas the training engine recomputes token probabilities in a separate forward pass, denoted π_{old} , under different numerical precision, kernels, and parallelization. Even for the same token under the same prefix, the two probabilities can differ:

$$\pi_{\text{rollout}}(a_t | s_t) \neq \pi_{\text{old}}(a_t | s_t). \quad (9)$$

Ignoring this mismatch introduces off-policy bias into the gradient estimator [15]. To mitigate this remaining mismatch, we apply importance-sampling rollout correction [34]. In principle, a sequence-level importance ratio [15] yields an unbiased estimator but can exhibit high variance over long trajectories. Since the mismatch between π_{rollout} and π_{old} is typically small, we adopt a token-level importance-sampling ratio, trading estimation bias for lower variance. Specifically, the loss of each training token is scaled by

$$w_t = \min(\exp(\log \pi_{\text{old}}(a_t | s_t) - \log \pi_{\text{rollout}}(a_t | s_t)), \bar{c}), \quad (10)$$

where $\log \pi_{\text{rollout}}$ is recorded by the inference engine at rollout time, $\log \pi_{\text{old}}$ is recomputed by the training engine before policy updates, and the truncation threshold $\bar{c}$ bounds the variance from outlier ratios.

3.3. Mix-Harness Training

Harnesses can differ substantially in their interaction protocols, tool interfaces, context-management strategies, and control flows. Because our framework connects each harness to the same model-serving boundary and recovered trajectory representation, different harnesses can be readily integrated into a unified training pipeline without modifying the underlying policy optimization procedure. This design supports direct and flexible training through any individual harness under a common interface.

Training through a single fixed harness, however, may encourage the policy to specialize to its tool-use conventions, context-management strategy, and execution workflow. Such specialization may limit broader generalization when the same task is executed through another harness with different interaction patterns. We therefore introduce *mix-harness training*, which jointly optimizes a shared policy using rollouts from multiple heterogeneous harnesses within the same training run.

For each task q with environment $\mathcal{E}_q$, we construct separate training instances $(q, \mathcal{E}_q, H_k)$ by pairing the task with different compatible harnesses H_k . Since the environment is uniquely determined by the task, we refer to each instance as a task–harness pair. Instances associated with different harnesses are randomly mixed within each training batch and jointly processed through the shared training pipeline.

The main additional consideration is rollout grouping. For group-based advantage estimation, rollouts within a group must remain comparable. We therefore define each optimization group by a task–harness pair rather than by the task alone. Rollouts of the same task under different harnesses may appear in the same batch, but their advantages are normalized within separate task–harness groups. This prevents harness-dependent interaction patterns and reward distributions from distorting relative advantage estimation, while allowing all groups to jointly update the shared policy.

We instantiate both individual-harness and mix-harness training using two representative harnesses, OpenClaw and Claude Code. OpenClaw is a general-purpose AI assistant harness supporting diverse workspace-grounded tasks, while Claude Code is a mature harness designed for long-horizon coding and terminal interaction. Detailed empirical results are provided in Section 4.3.

3.4. Safeguards for Reliable Training

Running RL through real harnesses and remotely provisioned sandboxes introduces failures and irregularities that arise from the execution infrastructure rather than the policy model itself. We employ several additional engineering safeguards to improve the reliability of rollout collection and training.

Robust Sandbox Execution and Fault Handling. Black-box RL relies on remotely provisioned sandboxes to execute real harnesses, making infrastructure-level failures such as execution timeouts, connection losses, and transient platform errors unavoidable yet unrelated to the policy model itself. If left unhandled, these failures can stall rollout collection or introduce invalid trajectories into training. To improve training robustness, we employ several fail-safe mechanisms. Long-horizon tasks are forcibly terminated once a wall-clock timeout is reached, while those that encounter explicit sandbox failures, such as HTTP errors or initialization failures, are discarded and excluded from both advantage estimation and policy optimization.

Buffered Pseudo-Streaming Parsing. Most black-box harnesses interact with agents through streaming generation and rely on incremental tool-call parsers to recover structured tool invocations online. In rare cases, however, incremental parsing may prematurely terminate a tool call because of parsing errors, even when the model generates a correct response. To address this issue, we adopt a *pseudo-streaming* parsing scheme. During generation, output tokens are buffered while synthetic Server-Sent Events (SSE) are continuously emitted to keep the streaming connection alive. After the entire turn completes, the buffered output is parsed once using a non-streaming parser, eliminating incremental parsing errors while preserving the streaming interface.

Complete Trajectory Capture via Settling. Since black-box harnesses execute asynchronously and may perform retries or fault handling in the background, the completion or failure of an external request does not guarantee that all trajectory records have been written. Collecting them immediately may therefore produce incomplete trajectories. Before trajectory assembly, we wait until the number of records remains unchanged across several checks and no records are pending. A fixed timeout prevents indefinite waiting.

4. Experiments

We evaluate the effectiveness, stability, and extensibility of our black-box RL framework. We first report overall performance under OpenClaw and Claude Code, then examine training dynamics across harnesses and optimization algorithms and evaluate mix-harness training. We further extend the framework to more challenging tasks from JobBench and OfficeQA, study the effect of cold-start initialization, and compare black-box RL with a white-box AgentLoop setting.

4.1. Overall Performance

4.1.1. Experimental Settings

Evaluation Datasets and Metrics. We evaluate all models using Pass@1 on ClawGym-Bench [4] and PinchBench [22] under a hybrid protocol that combines code-based verification with rubric-based judgment. On ClawGym-Bench, tasks whose verifier consists solely of code checks are scored by that verifier alone, whereas tasks whose verifier combines code checks with rubrics are scored as a weighted sum of the two, with weights 0.7 and 0.3, respectively. For PinchBench, we take the task set released on April 10, 2026, discard the multimodal tasks, and retain 30 tasks, each scored with the verifier provided by the benchmark. All rubric-based judgments use GPT-5.4 [18], with the prompt in Appendix A.

Evaluated Models. We take as baselines the Qwen3 series [33] and ClawGym-Agents [4]. For black-box RL, we train Qwen3-8B and Qwen3-30A3B under two representative harnesses, OpenClaw [19] and Claude Code [2], resulting in two model families, denoted as ClawII-OC and ClawII-CC, respectively. The

training tasks are drawn from ClawGym-SynData [4]. Due to data availability, ClawII-OC is initialized from ClawII-Cold, a model family obtained through a lightweight cold start on ClawGym-SynData, whereas ClawII-CC is trained directly from the corresponding base models. All models are evaluated through black-box rollouts using the corresponding harness. The maximum context window is set to 64K tokens, with longer trajectories handled by the context management of the corresponding harness.

4.1.2. Results

Table 1. Performance comparison of different models on ClawGym-Bench and PinchBench. The best and second-best Pass@1 results within each model group are highlighted in **bold** and underlined.

Model	Pinch-Bench	ClawGym-Bench						
		Product. & Collab.	Systems & Auto.	Analysis & Reason.	Content & Domain	Planning & Knowl.	Software Dev.	Avg.
OpenClaw as Harness								
Qwen3-8B	54.50	37.46	29.06	30.40	41.12	44.47	30.69	35.02
Qwen3-32B	49.40	40.68	36.21	37.84	47.16	49.29	33.11	40.32
Qwen3-30A3B	55.60	42.47	42.09	45.04	51.95	45.98	46.24	45.11
Qwen3-235A23B	60.60	<u>53.66</u>	52.27	47.18	61.39	67.33	49.23	54.48
ClawGym-8B	75.70	49.47	46.83	46.35	55.37	52.29	54.78	50.24
ClawGym-30A3B	<u>86.00</u>	52.98	50.97	64.64	<u>61.46</u>	57.90	<u>56.13</u>	<u>56.82</u>
ClawII-Cold-8B	71.29	48.80	42.49	45.06	51.39	54.74	42.47	47.06
ClawII-Cold-30A3B	75.61	54.34	47.61	52.58	53.42	60.64	48.88	52.64
ClawII-OC-8B	77.44	53.17	<u>55.99</u>	51.28	59.60	61.68	50.83	54.98
ClawII-OC-30A3B	87.32	62.75	59.81	<u>64.09</u>	62.18	<u>63.12</u>	66.71	62.62
Claude Code as Harness								
Qwen3-8B	27.40	16.19	13.94	20.26	25.37	23.62	14.65	18.54
Qwen3-32B	53.27	33.34	25.63	27.68	33.03	27.85	28.30	29.37
Qwen3-30A3B	54.14	38.75	33.06	32.99	45.43	41.62	32.26	37.06
Qwen3-235A23B	<u>62.47</u>	<u>41.74</u>	<u>40.53</u>	<u>43.70</u>	<u>50.35</u>	<u>49.03</u>	<u>54.90</u>	<u>45.59</u>
ClawII-CC-8B	61.21	41.27	38.19	41.65	49.33	44.61	39.37	42.05
ClawII-CC-30A3B	71.42	47.84	45.59	48.88	57.05	56.93	62.93	51.87

Table 1 shows the performance of our black-box RL framework. We make the following observations:

- *Black-Box RL Delivers Substantial Gains.* Black-box RL substantially improves the models used to initialize training. With the 30A3B backbone, ClawII-OC and ClawII-CC outperform their respective initial policies by 9.98 and 14.81 points on ClawGym-Bench under OpenClaw and Claude Code, respectively. Under OpenClaw, ClawII-OC-30A3B further exceeds the ClawGym-30A3B SFT baseline by 5.80 points. Consistent gains on PinchBench further show that these improvements transfer to an external benchmark.
- *Consistent Gains across Heterogeneous Harnesses.* Black-box RL remains effective across two structurally distinct harnesses, OpenClaw and Claude Code. When each model is trained and evaluated within the corresponding harness, it improves the 8B and 30A3B backbones by 7.92 and 9.98 points on OpenClaw, and by 23.51 and 14.81 points on Claude Code. The resulting 30A3B models further outperform Qwen3-235A23B by 8.14 and 6.28 points in the two settings, demonstrating the framework’s applicability across heterogeneous harnesses.
- *Robustness to Initialization Strategy.* Black-box RL succeeds both with and without a cold-start stage. ClawII-OC is initialized from ClawII-Cold, whereas ClawII-CC is optimized directly from the

corresponding base models. Both families improve consistently across model scales and task categories, indicating that a specialized warm-up stage is not required for effective training. Nevertheless, under the OpenClaw harness, introducing a lightweight cold start can provide a stronger initialization and lead to further performance gains. The analysis of the effect of cold-start initialization is provided in Section 4.5.

4.2. Training Dynamics Across Harnesses

We study the training dynamics of black-box RL to examine whether the proposed framework supports stable policy optimization across heterogeneous harnesses and RL algorithms. Using Qwen3-30A3B, we train through both OpenClaw and Claude Code with critic-based PPO and critic-free GRPO. As described in Section 3.2.3, both algorithms are adapted to optimize over the recovered multi-trajectory tree structure.

All experiments use tasks from ClawGym-SynData, with periodic evaluation on ClawGym-Bench. Due to the data availability constraint described earlier, the OpenClaw runs are initialized from a cold-started policy model, whereas the Claude Code runs begin directly from the corresponding base model. GRPO uses batches of 32 tasks with 8 rollouts per task, while PPO uses batches of 256 tasks with 1 rollout per task, resulting in the same rollout budget per update. Under this matched rollout budget, PPO covers a larger number of unique tasks per update, whereas GRPO allocates multiple rollouts to each task for group-based advantage estimation. PPO additionally requires a value model, which is initialized through a dedicated value-pretraining stage before RL optimization.

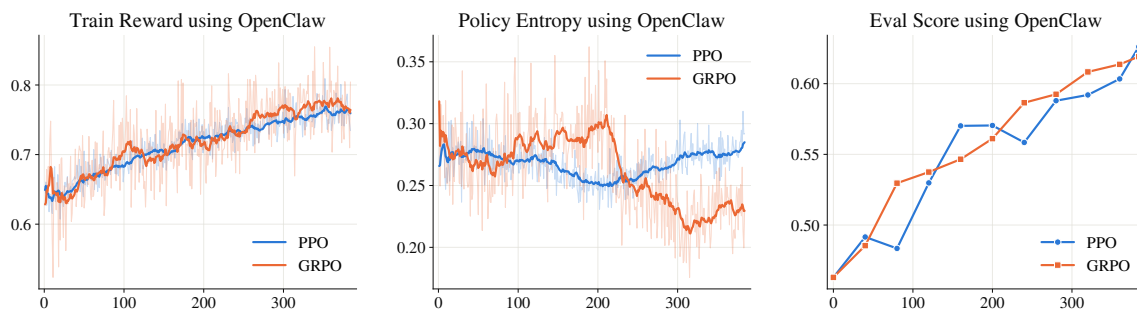


Figure 2. Training dynamics of PPO and GRPO with OpenClaw as the rollout harness.

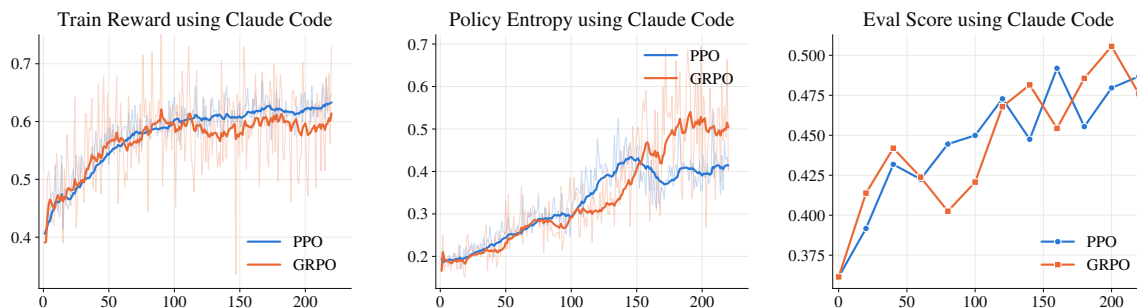


Figure 3. Training dynamics of PPO and GRPO with Claude Code as the rollout harness.

As shown in Figures 2 and 3, both PPO and GRPO maintain stable optimization over approximately 200–400 steps, showing clear upward trends in training reward and downstream evaluation performance while achieving broadly comparable final results. PPO generally exhibits smoother entropy dynamics, whereas GRPO shows larger variation, including a late-stage entropy decline under OpenClaw. Under Claude Code, both algorithms exhibit higher entropy than under OpenClaw before stabilizing. This difference may partly stem from the absence of cold-start initialization in the Claude Code runs, rather than from the harness itself. We examine this effect in Section 4.5. These results demonstrate that our

black-box RL framework can reliably optimize general agent models through heterogeneous harnesses under both critic-based PPO and critic-free GRPO.

4.3. Mix-Harness Training

We further investigate whether a single model can be jointly optimized through multiple heterogeneous harnesses within a unified black-box RL pipeline. Starting directly from Qwen3-30A3B without cold-start initialization, we perform GRPO training with both OpenClaw and Claude Code. As described in Section 3.3, for each task, we instantiate two task-harness pairs by executing the same task environment through OpenClaw and Claude Code, respectively. Task-harness pairs from both execution systems are randomly mixed within the same training batch, while GRPO grouping and relative-advantage normalization are performed independently for each pair. Thus, rollouts generated through different harnesses can coexist in the same batch but do not share the same group statistics or advantage baseline. The resulting gradients from all task-harness groups are aggregated within the same optimization step to jointly update a single policy model. Both the individual-harness and mix-harness settings use the same rollout batch configuration of 32 task-harness instances with 8 rollouts per instance, with periodic evaluation on ClawGym-Bench.

As shown in Figure 4, the mixed model exhibits training rewards comparable to those of the corresponding individual-harness models. Under OpenClaw, it gradually narrows the initial gap toward the end of training, while under Claude Code, its training reward remains on par with or higher than that of the Claude-Code-only model. A similar pattern is observed in downstream evaluation, where the mixed model matches or slightly outperforms the corresponding individual-harness models under both harnesses. Overall, combining rollouts from heterogeneous harnesses introduces no evident training instability or systematic performance degradation. These results demonstrate that learning signals from heterogeneous harnesses can be jointly leveraged to optimize a shared policy model within our unified black-box RL framework, without evident performance degradation under either execution system.

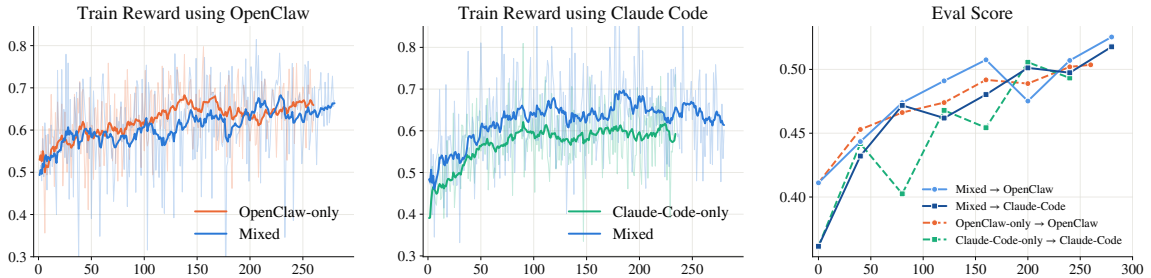


Figure 4. Comparison between single-harness and mix-harness RL training.

4.4. Training on More Challenging Tasks

To further evaluate the generality of our unified framework, we extend black-box RL to more challenging and structurally diverse task settings from JobBench [14] and OfficeQA [20]. JobBench targets professional workflows over heterogeneous workspaces containing diverse file formats, including images, databases, and native Office files, requiring agents to interpret and integrate information across multiple sources to complete complex tasks. OfficeQA instead emphasizes answer-centric reasoning over a large document corpus, where agents must retrieve relevant evidence, perform multi-step analysis or computation, and produce a verifiable final answer rather than a workspace artifact. Together, the two benchmarks cover substantially different forms of complex agent interaction, from artifact-oriented workspace execution to document-grounded analytical reasoning.

Following their respective task formats, we synthesize **JobBench-style** and **OfficeQA-style** training tasks and perform black-box RL on Qwen3-30A3B with Claude Code as the rollout harness. Under our unified

framework, a new task can be directly incorporated into training by simply formulating it as a triplet of an instruction, an initialized workspace, and a verifier, without requiring task-specific modifications to the RL pipeline. As shown in Figures 5 and 6, evaluation performance improves from 20.46 to 27.20 on JobBench-Easy and from 8.53 to 21.54 on OfficeQA-Full. Training rewards also maintain clear and stable upward trends throughout optimization. These results demonstrate that our framework is not tied to a particular task format or evaluation paradigm: the same unified black-box RL pipeline can be readily extended to substantially different and more demanding task distributions, while continuing to deliver effective and stable policy optimization.

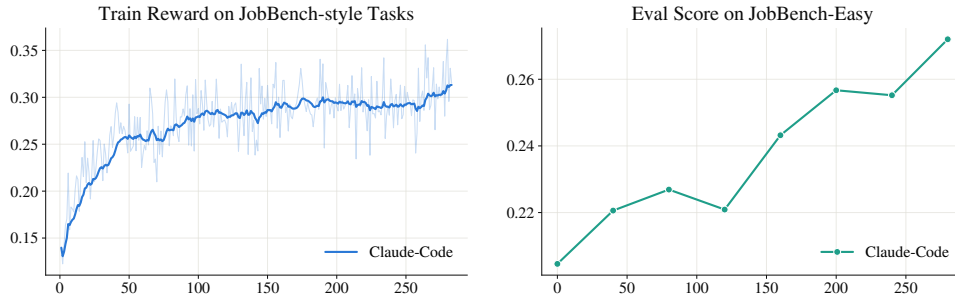


Figure 5. Training dynamics with Claude Code on JobBench-style tasks.

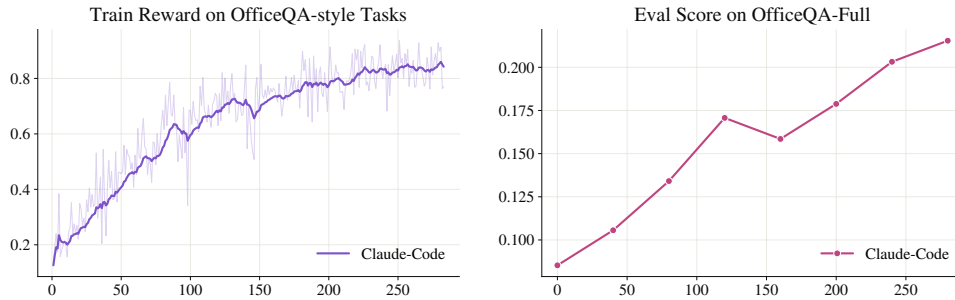


Figure 6. Training dynamics with Claude Code on OfficeQA-style tasks.

4.5. Effect of Cold-Start Initialization

We investigate the effect of cold-start initialization on black-box RL for general agent tasks using Qwen3-30A3B, with OpenClaw as the rollout harness and GRPO as the optimization algorithm. Specifically, we compare two initialization strategies: lightweight supervised training on a subset of trajectories from ClawGym-SynData, and direct initialization from the base model.

As shown in Figure 7, both settings benefit from RL, but the cold-started model consistently exhibits more favorable training dynamics. It starts from a substantially higher training reward and follows a smoother upward trajectory throughout optimization. Its policy entropy also remains relatively stable, whereas direct training from the base model exhibits larger fluctuations and a pronounced late-stage decline in entropy, indicating more volatile entropy dynamics and potentially less stable exploration. Moreover, the cold-started model achieves consistently stronger downstream performance. These results indicate that lightweight cold-start training provides a better behavioral prior and improves optimization stability in this setting, although direct RL training from the base model remains effective.

4.6. Comparison with White-Box AgentLoop RL

Unlike black-box RL, which interacts with an opaque harness and observes only model calls at the serving boundary, white-box RL directly exposes the complete interaction process through an explicitly

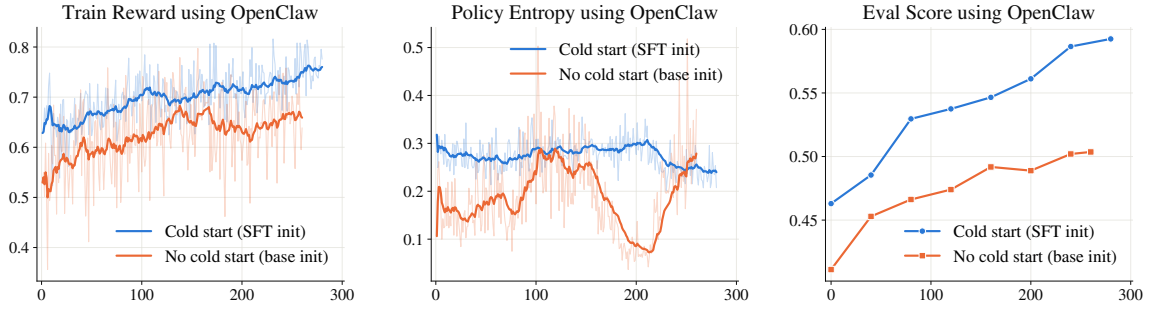


Figure 7. Effect of cold-start initialization on OpenClaw black-box RL.

constructed agent loop [29], including the system prompt, tool interface, observation representation, context management, and workflow. These components can be independently designed and assembled to form different agent loops.

During rollout, model generation, tool execution, observation return, and context update are directly recorded as a complete multi-turn trajectory. This makes it possible to optimize directly on the exact interaction traces produced by the agent loop. Specifically, we instantiate a white-box agent loop with five categories of basic tools: bash, search, fetch, code interpreter, and file operations. We perform RL training through this agent loop in sandbox-based environments, and evaluate the resulting policy model under both the same white-box agent loop and an external black-box harness to assess in-loop performance and white-to-black generalization transfer. All experiments follow the same datasets and evaluation settings as the black-box counterpart, ensuring a fair comparison across training paradigms.

Table 2. Cross-harness Pass@1 performance on ClawGym-Bench under the white-box AgentLoop and OpenClaw harnesses. ClawII-OC-30A3B and WhiteBox-30A3B are trained using OpenClaw-based black-box RL and white-box AgentLoop RL, respectively. The best and second-best results within each evaluation harness are highlighted in **bold** and underlined.

Model	Product. & Collab.	Systems & Auto.	Analysis & Reason.	Content & Domain	Planning & Knowl.	Software Dev.	Avg.
White-Box AgentLoop as Harness							
Qwen3-32B	28.79	24.38	22.34	27.88	27.25	26.69	26.43
Qwen3-30A3B	44.22	35.93	36.85	39.76	49.55	48.09	41.69
ClawII-OC-30A3B	<u>50.22</u>	<u>46.09</u>	<u>51.64</u>	<u>53.55</u>	<u>59.11</u>	<u>51.72</u>	<u>51.37</u>
WhiteBox-30A3B	62.49	53.36	58.75	61.35	68.50	57.33	59.90
OpenClaw as Harness							
Qwen3-32B	40.68	36.21	37.84	47.16	49.29	33.11	40.32
Qwen3-30A3B	42.47	42.09	45.04	51.95	45.98	46.24	45.11
ClawII-OC-30A3B	62.75	59.81	64.09	62.18	63.12	66.71	62.62
WhiteBox-30A3B	<u>49.13</u>	<u>44.60</u>	<u>51.10</u>	<u>50.36</u>	<u>57.91</u>	<u>53.01</u>	<u>50.33</u>

Effectiveness under the white-box agentloop. White-box agentloop RL yields substantial gains when evaluated under the same interaction framework used for training. As shown in Table 2, WhiteBox-30A3B achieves an average score of 59.90, improving over its Qwen3-30A3B initialization by 18.21 points and outperforming the black-box-trained ClawII-OC-30A3B by 8.53 points. The improvement is consistent across all six task categories. Figure 8 further shows that both GRPO and PPO exhibit stable optimization dynamics under the white-box agentloop, with training reward and evaluation performance improving over the course of training. These results demonstrate that the explicitly constructed agent loop provides

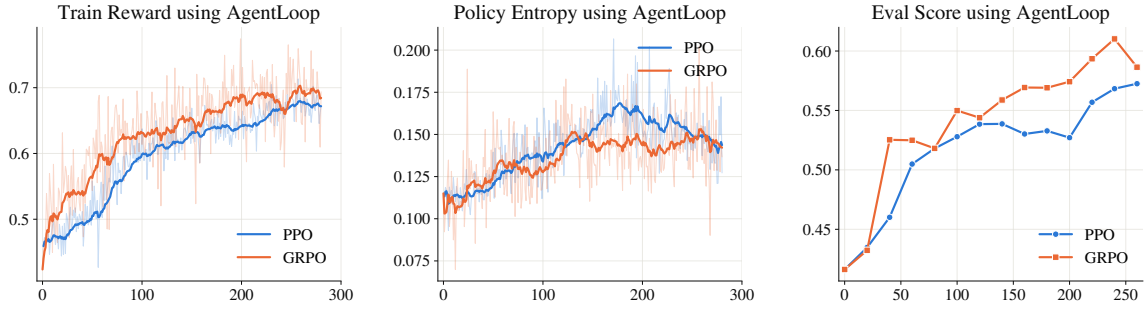


Figure 8. Training dynamics of white-box AgentLoop RL with GRPO and PPO. The curves report training reward, policy entropy, and evaluation score over the course of training.

an effective and stable environment for policy optimization across different RL algorithms.

White-to-black transfer. The policy model trained with the white-box agentloop also transfers to the external OpenClaw harness without additional training. WhiteBox-30A3B reaches an average score of 50.33 under OpenClaw, exceeding the original Qwen3-30A3B model by 5.22 points and improving performance in five of the six categories. This transfer suggests that different harnesses share a common set of underlying agentic capabilities that can be acquired through white-box training. However, WhiteBox-30A3B still falls behind ClawII-OC-30A3B, which is trained directly under OpenClaw and achieves 62.62, revealing that training under a simple white-box agentloop may still be insufficient to fully capture the harness-specific interaction patterns required by an unseen black-box system.

5. Conclusion

In this paper, we establish black-box reinforcement learning as a practical and scalable approach for training general autonomous agents directly through complex deployment harnesses. The native harness is treated as an unmodified and opaque rollout engine, thereby decoupling policy optimization from harness execution. We develop an end-to-end infrastructure that isolates rollouts in temporary sandboxes, captures faithful model behavior at the serving boundary, reconstructs forked multi-turn trajectories with prefix trees, and preserves training–inference consistency. Building on this infrastructure, we adapt both PPO and GRPO to tasks that may yield multiple trajectories and further extend the formulation to mix-harness training, enabling a single model to jointly learn from heterogeneous harnesses within a unified optimization pipeline. Experiments on two structurally distinct harnesses, OpenClaw and Claude Code, validate the effectiveness and stability of this formulation. With Qwen3-30A3B, black-box RL improves Pass@1 by 9.98 and 14.81 points on ClawGym-Bench and by 11.71 and 17.28 points on PinchBench, while remaining stable over 200–400 optimization steps. The same framework also extends to more challenging task settings from JobBench and OfficeQA, where black-box RL continues to improve downstream performance. These results demonstrate that complex deployment harnesses can serve as effective training interfaces, providing a practical foundation for optimizing generalist agents directly within the execution systems in which they operate.

In future work, we will explore incorporating auxiliary interactions, such as compaction and subagent trajectories, into policy optimization rather than excluding them from training. We also plan to extend our framework to broader general agent tasks and more diverse execution settings to further assess its scalability and generality.

References

- 1 AgentScope-AI. QwenPaw: Your personal ai assistant. <https://github.com/agentscope-ai/QwenPaw>, 2026. GitHub repository. Accessed: 2026-04-29.
- 2 Anthropic. Claude Code: Ai-powered coding assistant for developers. <https://claude.com/product/claude-code>, 2026. Product page. Accessed: 2026-04-29.
- 3 Fei Bai, Yingqian Min, Beichen Zhang, Zhipeng Chen, Xin Zhao, Lei Fang, Zheng Liu, Zhongyuan Wang, and Hongteng Xu. Towards effective code-integrated reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 30022–30030, 2026.
- 4 Fei Bai, Huatong Song, Shuang Sun, Daixuan Cheng, Yike Yang, Chuan Hao, Renyuan Li, Feng Chang, Yuan Wei, Ran Tao, et al. Clawgym: A scalable framework for building effective claw agents. *arXiv preprint arXiv:2604.26904*, 2026.
- 5 Chaithanya Bandi, Razvan-Gabriel Dumitru, Ben Hertzberg, Divyansh Agarwal, Geobio Boo, Tejas Polakam, Sami Hassaan, Jeff Da, HiJae Kim, Vipul Gupta, et al. Mcp-atlas: A large-scale benchmark for tool-use competency with real mcp servers. *arXiv preprint arXiv:2602.00933*, 2026.
- 6 Zhipeng Chen, Yingqian Min, Beichen Zhang, Jie Chen, Jinhao Jiang, Daixuan Cheng, Wayne Xin Zhao, Zheng Liu, Xu Miao, Yang Lu, et al. An empirical study on eliciting and improving r1-like reasoning models. *arXiv preprint arXiv:2503.04548*, 2025.
- 7 Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, Evan Mays, Rachel Dias, Marwan Aljubei, Mia Glaese, Carlos E. Jimenez, John Yang, Leyton Ho, Tejal Patwardhan, Kevin Liu, and Aleksander Madry. Introducing swe-bench verified, August 2024. URL <https://openai.com/index/introducing-swe-bench-verified/>.
- 8 Databricks. Officeqa: A grounded reasoning benchmark, 2025.
- 9 Quentin Gallouédec and Kashif Rasul. Agentic rl: Token-in, token-out done right, May 2026. URL <https://huggingface.co/blog/huggingface/tito>. Hugging Face Blog.
- 10 HKUDS. NanoBot: The ultra-lightweight personal ai agent. <https://github.com/HKUDS/nanobot>, 2026. GitHub repository. Accessed: 2026-04-29.
- 11 Liangmeng Huang, Qingchuan Li, Hongwei Xue, Shilin Yan, and Dressage Contributors. Dressage: Scalable RL for any agent and any sandbox. <https://github.com/Accio-Lab/Dressage>, 2026.
- 12 Fengxiang Li, Han Zhang, Haoyang Huang, Jinghui Wang, Jinhua Hao, Kun Yuan, Mengtong Li, Minglei Zhang, Pengcheng Xu, Wenhao Zhuang, et al. Kat-coder-v2 technical report. *arXiv preprint arXiv:2603.27703*, 2026.
- 13 Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. *arXiv preprint arXiv:2510.25726*, 2025.
- 14 Yuetai Li, Yichen Feng, Zhangchen Xu, Zixian Ma, Kaiyuan Zheng, Fengqing Jiang, Xinghua Sun, Rulin Shao, Zichen Chen, Yue Huang, Xinyang Han, Brian Lee, Kayla Xu, Shenglai Zeng, Hang Hua, Xiangliang Zhang, Basel Alomair, Ranjay Krishna, Luke Zettlemoyer, Pang Wei Koh, Bhaskar Ramasubramanian, Luyao Niu, Xiang Yue, and Radha Poovendran. Jobbench: Aligning agent work with human will, 2026. URL <https://arxiv.org/abs/2605.26329>.

- 15 Jiakai Liu, Yingru Li, Yuqian Fu, Jiawei Wang, Qian Liu, and Yu Shen. When speed kills stability: Demystifying RL collapse from the training-inference mismatch, September 2025. URL <https://richardli.xyz/rl-collapse>.
- 16 Mike A Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Jenia Jitsev, Marianna Nezhurina, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Kumar Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Kwesi Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a7Qa4CcHak>.
- 17 OpenAI. Codex: Ai coding partner from openai. <https://openai.com/codex/>, 2026. Product page. Accessed: 2026-04-29.
- 18 OpenAI. Introducing GPT-5.4. <https://openai.com/index/introducing-gpt-5-4/>, March 2026. Official release announcement. Accessed: 2026-04-29.
- 19 OpenClaw. OpenClaw: Personal ai assistant. <https://github.com/openclaw/openclaw>, 2026. GitHub repository. Accessed: 2026-04-29.
- 20 Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. Officeqa pro: An enterprise benchmark for end-to-end grounded reasoning, 2026. URL <https://arxiv.org/abs/2603.08655>.
- 21 Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simón Posada Fishman, Marwan Aljubei, Phoebe Thacker, Laurance Fauconnet, Natalie S. Kim, Patrick Chao, Samuel Miserendino, Gildas Chabot, David Li, Michael Sharman, Alexandra Barr, Amelia Glaese, and Jerry Tworek. Gdpval: Evaluating ai model performance on real-world economically valuable tasks, 2025. URL <https://arxiv.org/abs/2510.04374>.
- 22 PinchBench. PinchBench: Real-world benchmarks for ai coding agents. <https://github.com/pinchbench/skill>, 2026. GitHub repository. Accessed: 2026-04-29.
- 23 Qwen Team and Alibaba Group Data Team. QwenClawBench: Real-user-distribution benchmark for openclaw agents, April 2026. URL github.com/SKYLENAGE-AI/QwenClawBench.
- 24 RUC-NLPIR. Awesome long-horizon agents. <https://github.com/RUC-NLPIR/Awesome-Long-Horizon-Agents>, 2026. GitHub repository.
- 25 John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

- 26 Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- 27 Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- 28 Huatong Song, Lisheng Huang, Shuang Sun, Jinhao Jiang, Ran Le, Daixuan Cheng, Guoxin Chen, Yiwen Hu, Zongchao Chen, Yiming Jia, et al. Swe-master: Unleashing the potential of software engineering agents via post-training. *arXiv preprint arXiv:2602.03411*, 2026.
- 29 Shuang Sun, Huatong Song, Lisheng Huang, Jinhao Jiang, Ran Le, Zhihao Lv, Zongchao Chen, Yiwen Hu, Wenyang Luo, Wayne Xin Zhao, et al. Swe-world: Building software engineering agents in docker-free environments. *arXiv preprint arXiv:2602.03419*, 2026.
- 30 Xinyu Tang, Han Peng, Guoxin Chen, Yuze Shi, Zitao Su, Peiyu Liu, Wayne Xin Zhao, Yawen Li, and Zhe Xue. Agent systems with harness engineering, 2026. URL <https://openreview.net/pdf?id=nM5tDHrQsx>.
- 31 Ruhan Wang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Yue Yu, Junyao Yang, Kishan Panaganti, Haitao Mi, Dongruo Zhou, et al. Harness handbook: Making evolving agent harnesses readable, navigable, and editable. *arXiv preprint arXiv:2607.13285*, 2026.
- 32 Binfeng Xu, Hao Zhang, Shaokun Zhang, Songyang Han, Mingjie Liu, Jian Hu, Shizhe Diao, Zhenghui Jin, Yunheng Zou, Michael Demoret, et al. Polar: Agentic rl on any harness at scale. *arXiv preprint arXiv:2605.24220*, 2026.
- 33 An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- 34 Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Your efficient rl framework secretly brings you off-policy rl training, August 2025. URL <https://fengyao.notion.site/off-policy-rl>.
- 35 Shunyu Yao, Jeffrey Zhao, Dian Yu, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*, 2022.
- 36 Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, et al. Claw-eval: Toward trustworthy evaluation of autonomous agents. *arXiv preprint arXiv:2604.06132*, 2026.
- 37 Xiao Yu, Baolin Peng, Ruize Xu, Hao Zou, Qianhui Wu, Hao Cheng, Wenlin Yao, Nikhil Singh, Zhou Yu, and Jianfeng Gao. Openforgerl: Train harness-native agents in any environment. *arXiv preprint arXiv:2607.21557*, 2026.
- 38 Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.

A. Evaluation Prompt

Rubric-Based Judge

System prompt:

You are a strict rubric-based evaluator. Use only the user prompt content. Do not call tools, browse, inspect files, or ask for more context. Your response may include concise analysis, but it must end with exactly one standalone JSON object with keys `scores` and `notes`.

User prompt:

You are grading an OpenClaw agent result.

You must not call, request, or simulate any tools. Do not browse, list directories, open files, inspect the workspace, or ask for more context. Grade only from the task, final output files, optional transcript evidence, and rubrics included in this prompt.

Before giving the final judgment, provide concise analysis explaining how the final outputs satisfy or fail each rubric criterion.

Your final judgment must end with exactly one standalone JSON object and nothing else after it. The final JSON object must contain exactly two keys: `scores` and `notes`.

- `scores` must map every rubric id (`criterion\_1`, `criterion\_2`, ...) to one numeric score chosen from that rubric's allowed score anchors.
- `notes` must be a concise string summarizing the main reasons for the assigned scores.

Do not include any overall score in the final JSON. Do not compute or report the final aggregated score. Score aggregation will be handled separately by post-processing.

User Task
{{USER_TASK}}

Final Output Files
{{FINAL_OUTPUT_FILES}}

Additional Changed Workspace Files
{{ADDITIONAL_CHANGED_WORKSPACE_FILES}}

Transcript
{{TRANSCRIPT_OPTIONAL}}

Rubric
{{RUBRIC}}